

COMPOSING AGENT ORCHESTRATIONS IS A ZAPPA-SZÉP PRODUCT A CONTAINER MODEL AND ITS DEGREE-TWO COMPOSITION OBSTRUCTION

MACBETH

ABSTRACT. An AI agent presents an *interface*: a set of queries it may issue and, for each, a set of admissible replies. This is exactly a polynomial functor, or container — a reading due to Spivak and, in the orchestration setting, to Ghani, with the interface/implementation mechanism (free-monad implementations composed along wiring diagrams, dependent-polynomial specifications) developed in dependent type theory by Aberlé. We take that reading seriously enough to ask what it says about *composing* orchestrations that share resources, and we find that the natural model is not a functor but a distributive law: welding two orchestrations into one interleaved system is a Zappa–Szép product $\mathcal{C} \bowtie \mathcal{D}$ of their handoff categories — the distributive-law construction for directed containers being Ahman–Uustalu’s. This reframing has teeth, because a Zappa–Szép product need not exist, and its obstruction is known: a degree-two cohomology class $[\omega] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$ on the handoff category with coefficients in the reply functor. Our main result works the smallest non-trivial case — a supervisor dispatching to workers, parametrized by a one-bit turn token — and proves that the obstruction class equals the token-mutation bit on the nose: the joint agent exists if and only if the worker fixes the token, and the sole obstructed regime is *unprotected re-entrancy*, a worker mutating the supervisor’s shared state during a pending call, which presents exactly as the nonzero generator of $H^2 \cong \mathbb{Z}/2$. The degree is the point: the multi-agent-systems literature places H^0 (consensus) and H^1 (identifiability) sheaf obstructions on the *communication* graph, asking whether agents agree; the container model places H^2 on the *handoff* category, asking the prior question of whether they compose at all.

1. INTRODUCTION

The problem. Modern AI systems are increasingly *orchestrations*: a supervisor agent dispatches subtasks to worker agents, a planner calls tools that call other tools, a proof assistant runs tactics that invoke sub-tactics. Each participant is a black box with a typed boundary — it accepts a query and returns one of a set of admissible replies — and the engineering discipline is entirely about how these boundaries are wired together. The question this note addresses is structural rather than behavioural: given two orchestrations that share resources, *when can they be run as a single consistent interleaved system*, and when is there a genuine obstruction to doing so?

Agents are containers. The starting point, due to Spivak in the polynomial-functor programme [1, 2] and articulated for agent orchestration by Ghani at Kodamai, is that an agent’s interface is a *container* (equivalently a polynomial functor). An agent that may issue queries drawn from a set S , and for each query s may receive a reply from a set $P(s)$, is the container $p = (S, P)$, or as a functor $p \cong \sum_{s \in S} y^{P(s)}$. An oracle call — issue a query, receive a reply — is a morphism $p \rightarrow y$ to the unit container; delegating from one agent to another is a container morphism, forward on queries and backward on replies. On this reading the monoidal structures of **Poly** become an orchestration vocabulary: sequential delegation, parallel composition, choice. This dictionary is not ours and we do not claim it (Section 2 grades each row); we take it as the setting.

Prior art: the interface/implementation mechanism. The mechanism that turns this dictionary into a working compositional framework is due to Aberlé [5], whose recent development we take as the immediate prior art. There, polynomial functors $p = (A, B)$ are program *interfaces*; an *implementation* is a morphism $p \Rightarrow \text{Free } q$ into the free monad on the assumed interface q (assume–guarantee reasoning: a bare lens is the special case of exactly one call, the free monad lifts it to zero/one/many), and implementations compose along Spivak’s operad of wiring diagrams by Kleisli composition ([5], Thm. 3.1). *Specifications* are dependent polynomials (C, D) over p — literally Hoare triples $\{C\} f \{D\}$ — and they compose along the same wiring diagrams (Thm. 5.3). The whole package is a strictly monoidal forgetful functor $\pi : \mathbf{Spec} \rightarrow \mathbf{Int}$ together with a lax monoidal presheaf of systems and a monoidal natural transformation (Def. 7.1). Everything in the present note’s dictionary that concerns *interfaces*, *free-monad implementations*, *wiring-diagram composition*, and *dependent-polynomial specifications* is Aberlé’s and is cited as such; our theorem-proving-as-container and orchestration-as-workflow readings are a *domain reinterpretation* of that mechanism, not a new mechanism. (Aberlé himself flags tactics and proof automation as future work.) What Aberlé does *not* treat, and what this note supplies, is the composition of two systems that *interact* over a shared resource: his only two-system construct is the unobstructed parallel sum. That gap is exactly where the contribution lives.

The gap. What the existing dictionary does *not* supply is a theory of when a composition is *possible*. Prior treatments model an orchestration as a functor over a category of interfaces — composition *along* a single fixed topology. But concurrency, a supervisor sharing workers between two plans, and re-entrant tool-calls are not composition along one topology; they are the *interleaving of two*. A functor cannot see the failure mode, because it presumes the composite already exists. We want a model in which the composite can fail to exist, together with an invariant that detects the failure.

The result. The model is a distributive law. A handoff topology — who may hand off to whom, staying put, concatenating handoffs — is a small category, equivalently a directed container in the sense of Ahman–Uustalu [3]. Interleaving two orchestrations $\mathcal{C}$ and $\mathcal{D}$ into a single joint agent is then a *Zappa–Szép product* (matched-pair product, strict factorization system) $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$. That a distributive law of directed containers assembles such a joint container — generalizing the Zappa–Szép product of monoids, its matched pair recording the two mutual actions — is a theorem of Ahman–Uustalu [4]; we take the construction from them and claim none of it. Such a product need not exist, however; the author’s pairwise criterion [6] characterizes *when* the complementary factor can be found, and by the author’s obstruction theorem [7] that existence is controlled by a degree-two cohomology class:

$$\text{the joint agent } \mathcal{C} \bowtie \mathcal{D} \text{ exists} \iff [\omega] = 0 \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}),$$

cohomology of the handoff category’s skeleton with coefficients in the reply (direction) functor $\mathcal{D}$. The cohomology itself is classical [10, 11, 12]; what is new is the identification of the orchestration composite with $\mathcal{C} \bowtie \mathcal{D}$ and hence of its obstruction with this class.

The method. We ground the claim on the smallest faithful instance rather than asserting the dictionary into a theorem. A supervisor dispatching to workers, with a one-bit turn token recording whose reply it awaits, is a small category $\mathcal{K}_\varepsilon$ parametrized by a single bit $\varepsilon \in \mathbb{Z}/2$: the worker’s re-entrant outcome sends the dispatch to $q \cdot \tau^\varepsilon$, so ε is exactly whether that outcome *mutates* the token ($\varepsilon = 1$) or *fixes* it ($\varepsilon = 0$). Interleaving $\mathcal{K}_\varepsilon$ with a second orchestration sharing the workers is a Zappa–Szép product. We prove *analytically* — verifying the freeness condition and the vertex-group hypothesis directly, computing the orbit category, the cochain complex, and the defect cocycle by hand — that the obstruction class equals the token-mutation bit on the nose,

$$[\omega(\mathcal{K}_\varepsilon)] = \varepsilon \cdot (\text{generator}) \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}) \cong \mathbb{Z}/2,$$

so the joint agent exists if and only if $\varepsilon = 0$. No enumeration of factorization systems and no machine-verified isomorphism is load-bearing; a parametrized script cross-checks every step. Two further regimes — independent supervisors over read-only workers (a trivial product) and two supervisors interleaving coherently into the non-abelian join S_3 — round out the picture at the grade of a *computed* illustration; the proved content is the dichotomy.

The consequences. Two payoffs. First, a named, structural failure mode: *unprotected re-entrancy* — the callee writing to the caller’s shared state during a pending call — is not a heuristic smell but an exact nonzero cohomology class on the handoff category, the categorical avatar of a flat bundle with nontrivial holonomy. Second, a clean position relative to the multi-agent-systems literature. That literature places cellular-sheaf obstructions of degree H^0 (consensus, Nash equilibria) and H^1 (identifiability) on the *communication* graph [13, 14, 15], asking whether agents currently agree. The container model places an obstruction one degree up, H^2 , on the *handoff* category, asking the prior question of whether the agents compose into a joint system at all. The two are complementary, and the degree axis is exact.

Outline. Section 2 sets out the agent-as-container dictionary, grading each row for provenance. Section 3 states the three black-box theorems and explains why composition is a Zappa–Szép product rather than a functor. Section 4 works the supervisor–worker instantiation and its four-regime table. Section 5 draws the degree-axis contrast with the sheaf-Laplacian methods and positions the result against the other 2026 categorical-orchestration accounts. Section 6 closes with the empirical-validation ask and open questions.

2. THE DICTIONARY, HONESTLY GRADED

We first fix the translation between orchestration concepts and container structure. The reader should treat this section as a glossary, not a sequence of theorems: most rows are established prior art, and we say so explicitly. The one row carrying this note’s contribution is flagged ★ and proved in Section 4.

Write **Cont** for the category of containers; a container $p = (S, P)$ has a *shape* set S and, for each shape s , a *position* set $P(s)$, and represents the polynomial functor $\sum_{s \in S} y^{P(s)}$.

Orchestration concept	Container structure	Provenance
An agent's interface	container $p = (S, P)$: S = queries it may issue, $P(s)$ = admissible replies to s	Spivak [1]
An oracle / LLM call	a morphism $p \rightarrow y$ (issue a query, receive one reply)	Ghani/Spivak
An agent's policy / dynamics	a p -coalgebra (Moore–Mealy lens) realising the interface	Spivak [1]
An agent's <i>implementation</i>	a Kleisli arrow $p \Rightarrow \text{Free } q$ into the free monad on its dependencies, composed along wiring diagrams	Aberlé [5]
A behavioural <i>specification</i>	a dependent polynomial (C, D) over p (pre/postconditions; a Hoare triple), verified compositionally	Aberlé [5]
Delegation between agents	a container morphism (forward on queries, backward amalgamating replies)	Ghani
A handoff <i>topology</i>	a small category = directed container $(S \triangleleft P, o, \downarrow, \oplus)$: objects are roles, $P(s)$ = admissible handoff paths out of s	Ahman–Uustalu [3]
Sequential delegation	composition product $\triangleleft$; its free monad = nested-call workflow trees (Aberlé's implementation structure)	Gambino–Kock; Aberlé [5]; Lean [9]
Keep-both-interfaces	the categorical product $\times$ (the unique pointwise structure)	prior art
Shared-schedule parallelism	the Dirichlet tensor $\otimes$ (Day convolution of $\times$)	Spivak [1]
Choice between agents	the coproduct $+$	prior art
Higher-order orchestration	a closed structure: $[q, r]$ = orchestrators consuming a q -agent, emitting an r -agent; $\mathbf{Cont}(p \otimes q, r) \cong \mathbf{Cont}(p, [q, r])$	Spivak [1]; Lean
The harness an agent runs in	a comonad on the topology; the cofree comonad is the maximal harness	Spivak
★ Interleaving two orchestrations	a distributive law / Zappa–Szép product $\mathcal{C} \bowtie \mathcal{D}$ (Ahman–Uustalu [4]), obstructed by $[\omega] \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D})$	construction A.–U.; obstruction: this note (§4)

Three remarks on the graded rows keep the reader honest.

The *interface* and *dynamics* rows are Spivak's interaction reading of **Poly** [1, 2]. The *implementation* content of the sequential and proof-search rows below — interfaces implemented by

free-monad(-on-**Poly**) Kleisli arrows and composed along wiring diagrams, with dependent polynomials as specifications — is Aberlé’s mechanism [5] (see the prior-art discussion above); we reinterpret it in two domains but claim none of the mechanism itself.

The *sequential* row is worth pausing on because it is the one place where the underlying free-monad structure is proved, machine-checked, and directly models a familiar orchestration pattern. The composition product $p \triangleleft q$ is “run p , and at each of its positions run a copy of q ”; iterating it freely produces the free monad on a container, whose elements are exactly *nested-call workflow trees* — a shape at the root, and for each of its positions a subtree. This free monad is Aberlé’s implementation structure; what we add is a machine check of its algebra: the three monoid laws (unit on both sides, associativity of grafting) are formalised in Lean with no remaining axioms beyond quotient soundness [9]. Under the proof-search reading, this is tactic composition; under the orchestration reading, it is a call tree.

The *proof-search* instance is the same model with the words changed: a theorem is a shape, the lemmas it needs are its positions, and a proof is an element of the free monad — a tree of theorems reduced to sub-theorems — with tactic composition realised as grafting. That the free-monad laws hold on the nose (loc. cit.) is what makes tactic composition associative. This is a domain reinterpretation of Aberlé’s verified-implementation framework, which itself names tactics and proof automation as future work.

3. WHY COMPOSITION IS A ZAPPA-SZÉP PRODUCT

We now state the three results the composition claim rests on. All are used as black boxes. The first is Ahman–Uustalu; so, crucially, is the construction the second theorem rests on — that a distributive law of directed containers is exactly a matched pair and assembles the joint Zappa–Szép container, generalizing the monoid case, is their theorem [4]. The author’s contribution in the second theorem is only the *existence* criterion (L) + (G) for the complementary factor; the cohomology in the third is classical.

Theorem 3.1 (Directed container $\cong$ small category; Ahman–Uustalu [3]). *A handoff topology — objects are roles, morphisms are admissible handoff paths, with identities (stay put) and composition (concatenate handoffs) — is a small category, hence a directed container $(S \triangleleft P, o, \downarrow, \oplus)$, and conversely. The two structures carry the same data. (Formalised in Lean in the author’s development.)*

Theorem 3.2 (Pairwise Zappa–Szép criterion [6]). *Let $\mathcal{K}$ be a small category and $\mathcal{D} \subseteq \mathcal{K}$ a wide subcategory. There is a wide subcategory $\mathcal{C} \subseteq \mathcal{K}$ making $(\mathcal{C}, \mathcal{D})$ a strict factorization system — equivalently a Zappa–Szép product $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$, equivalently a distributive law $\lambda : \mathcal{D}\mathcal{C} \Rightarrow \mathcal{C}\mathcal{D}$ satisfying the matched-pair axioms — if and only if*

- (L) *every hom-presheaf $\text{Hom}_{\mathcal{K}}(-, b)$ is free as a right $\mathcal{D}$ -set, and*
- (G) *free bases can be chosen closing into a wide subcategory.*

Theorem 3.3 (The closure obstruction is an H^2 class [7]). *Under the standard hypothesis (abelian vertex groups, trivial induced action), (G) holds if and only if a single cohomology class vanishes,*

$$(G) \iff [\omega] = 0 \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}),$$

cohomology of the skeleton of the handoff category with coefficients in the direction (reply) functor $\mathcal{D}$. The relevant cohomology is classical: existence is Rosebrugh–Wood [10], the abelian classification is Baues–Wirsching [11], and the non-abelian case is Pirashvili’s Schreier theory [12]. We cite it; we do not reprove it.

Functor versus distributive law. Here is the conceptual load of Theorem 3.2, and the reason it is the right model. A functor between handoff categories composes *along* a single topology: it presumes one fixed order of events and transports structure through it. But two orchestrations

that share resources do not present one topology; each has its own, and running them together requires *interleaving* the two. A distributive law $\lambda : \mathcal{DC} \Rightarrow \mathcal{CD}$ is exactly the data of a coherent interleaving — a rule for pushing a $\mathcal{D}$ -move past a $\mathcal{C}$ -move — and a Zappa–Szép product $\mathcal{C} \bowtie \mathcal{D}$ is the joint category it generates; that a distributive law of directed containers assembles this joint container is Ahman–Uustalu’s [4]. This is what concurrency, a supervisor sharing workers between two plans, and re-entrant tool-calls actually require, and it is strictly more than a functor can express: a functor always exists, whereas a distributive law can fail to. Theorem 3.3 says the failure is measured in a single cohomological degree.

Remark 3.4 (Scope). Theorems 3.1–3.3 are established. The novelty claimed here is the *identification*: that the joint system of two shared-resource orchestrations is $\mathcal{C} \bowtie \mathcal{D}$, so that its non-existence is exactly the nonvanishing of $[\omega]$. We ground that identification, rather than assert it, in Section 4.

4. THE WORKED INSTANTIATION

We exhibit the smallest orchestration in which the obstruction is non-trivial and *prove*, analytically, that its obstruction class equals a single design bit. The point is to move the starred dictionary row from a proposed reading to a theorem: the supervisor–worker family below *is* a category to which Theorems 3.1–3.3 apply, and its composition obstruction *is* a concrete H^2 generator, uniformly across the bit. Full computational detail is in the author’s analytic proof note [8]; we give the spine here.

The parametrized category. Fix handoff roles S (supervisor), W (a worker running), R (a return point). The supervisor carries a one-bit *turn token*; toggling it is an involution τ with $\tau^2 = \text{id}$, so $\text{End}(S) = \{\text{id}_S, \tau\} \cong \mathbb{Z}/2$. The generating handoffs are

$$p, p\tau : S \rightarrow W \quad (\text{dispatch in turn-state } 0 \text{ resp. } 1), \quad s, s_2 : W \rightarrow R \quad (\text{two worker outcomes}),$$

with composites $q, q\tau : S \rightarrow R$ and right $\mathbb{Z}/2$ -action $p \cdot \tau = p\tau$, $q \cdot \tau = q\tau$. The prescribed right factor is the supervisor-internal (token) part,

$$\mathcal{D} = \{\text{id}_S, \tau, \text{id}_W, \text{id}_R\},$$

a wide subcategory with vertex groups $G_S \cong \mathbb{Z}/2$ and $G_W = G_R = 0$; the left factor $\mathcal{C}$ — the genuine dispatch/handoff part — is what we solve for. Everything turns on one composite: how a worker outcome, run after a dispatch, sits relative to the token.

Definition 4.1 (The family $\mathcal{K}_\varepsilon$). For $\varepsilon \in \mathbb{Z}/2$ let $\mathcal{K}_\varepsilon$ be the resulting category with the single composite fixed by

$$s \circ p = q, \quad s_2 \circ p = q \cdot \tau^\varepsilon,$$

where s is “the worker returns normally” and s_2 is “the worker re-enters” (calls back into the supervisor before returning). Thus $\varepsilon = [s_2 \circ p = q\tau]$ records whether the re-entrant outcome *mutates* the token ($\varepsilon = 1$, *unprotected re-entry*) or *fixes* it ($\varepsilon = 0$, *state-protected re-entry / a lock*). Write $\mathcal{K}_{\text{ok}} := \mathcal{K}_0$ and $\mathcal{K}_{\text{bug}} := \mathcal{K}_1$.

Each $\mathcal{K}_\varepsilon$ is a genuine category with $\mathcal{D}$ wide (associativity reduces to right τ -equivariance of the two length-two composites; see [8]). The reading is exact: re-entrancy is the callee mutating shared caller state during a pending call, and here that is precisely the s_2 -branch dragging the turn token when $\varepsilon = 1$.

The obstruction class equals the token bit.

Theorem 4.2 (The obstruction is the token-mutation bit). *For each $\varepsilon \in \mathbb{Z}/2$, the family $\mathcal{K}_\varepsilon$ with distinguished wide subcategory $\mathcal{D}$ satisfies condition (L) and the hypothesis of Theorem 3.3, the skeleton $\text{Sk}_{\mathcal{C}}$ is the branch $S \rightarrow W \rightrightarrows R$ (independent of ε) with $H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}) \cong \mathbb{Z}/2$, and*

$$[\omega(\mathcal{K}_\varepsilon)] = \varepsilon \cdot (\text{generator}) \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}).$$

Consequently the joint agent $\mathcal{C} \bowtie \mathcal{D}$ exists — equivalently a distributive law $\lambda : \mathcal{D}\mathcal{C} \Rightarrow \mathcal{C}\mathcal{D}$ — if and only if $\varepsilon = 0$, i.e. iff the worker’s outcome fixes the supervisor’s token. For $\varepsilon = 1$ (unprotected re-entry) the obstruction is the nonzero generator and no serialization exists.

Proof. (L) and the hypothesis. The right $\mathcal{D}$ -action is precomposition; it is the regular $\mathbb{Z}/2$ -action on $\mathcal{S}$ -sourced arrows and trivial elsewhere. Decomposing into orbits, $\text{Hom}(-, \mathcal{W}) = \{p, p\tau\} \sqcup \{\text{id}_{\mathcal{W}}\}$ and $\text{Hom}(-, \mathcal{R}) = \{q, q\tau\} \sqcup \{s\} \sqcup \{s_2\} \sqcup \{\text{id}_{\mathcal{R}}\}$; since $p \neq p\tau$ and $q \neq q\tau$, every $\mathcal{S}$ -apex orbit is a free $\mathbb{Z}/2$ -torsor and every other orbit is a singleton, so each hom-presheaf is free — and this uses only $p \neq p\tau, q \neq q\tau$, so it is independent of ε (the value $q\tau^\varepsilon$ stays inside the orbit $\{q, q\tau\}$ either way). The hypothesis of Theorem 3.3 holds because the only nontrivial vertex group $G_{\mathcal{S}} = \mathbb{Z}/2$ sits at a source (nothing maps into $\mathcal{S}$ but its own endomorphisms), so the induced left action is trivial.

The class. The orbit category $\text{Sk}_{\mathcal{C}}$ has non-identity arrows $[p] : \mathcal{S} \rightarrow \mathcal{W}, [q] : \mathcal{S} \rightarrow \mathcal{R}, [s], [s_2] : \mathcal{W} \rightarrow \mathcal{R}$ with $[s] * [p] = [s_2] * [p] = [q]$ (both because $q, q\tau$ share an orbit) — the same branch for both ε . Every restriction map of the coefficient presheaf $\mathcal{D}$ vanishes (each generator targets $\mathcal{W}$ or $\mathcal{R}$, where $G = 0$). The normalized complex has $C^1 \cong C^2 \cong (\mathbb{Z}/2)^2$ (coordinates $h([p]), h([q])$ resp. $\omega([s], [p]), \omega([s_2], [p])$) and $C^3 = 0$, so $Z^2 = C^2$; the coboundary is $(\delta^1 h)([s], [p]) = (\delta^1 h)([s_2], [p]) = h([p]) - h([q])$, whence $B^2 = \langle (1, 1) \rangle$ (the diagonal) and $H^2 \cong \mathbb{Z}/2$. The natural transversal $T = \{p, q, s, s_2\}$ has defect $\omega_T([s], [p]) = \text{id}_{\mathcal{S}}$ (since $s \circ p = q \circ \text{id}_{\mathcal{S}}$) and $\omega_T([s_2], [p]) = \tau^\varepsilon$ (since $s_2 \circ p = q \circ \tau^\varepsilon$), i.e. $\omega_T = (0, \varepsilon)$; its class is 0 for $\varepsilon = 0$ and the generator $(0, 1) \notin \langle (1, 1) \rangle$ for $\varepsilon = 1$. The dichotomy then chains Theorems 3.2–3.3: $\mathcal{C} \bowtie \mathcal{D}$ exists $\iff (G) \iff [\omega] = 0 \iff \varepsilon = 0$. See [8] for the complete verification. $\square$

Remark 4.3 (No enumeration, no machine isomorphism). The argument is analytic: freeness, the hypothesis, the orbit category, the complex, and the defect are all computed by hand, and the class is read off uniformly in ε . A parametrized script cross-checks every step (and, independently of the proof, confirms $\#SFS = 2, 0$ for $\varepsilon = 0, 1$); the isomorphism of $\mathcal{K}_1$ with the ten-morphism rigid twist of [7] is now a corollary rather than a load-bearing machine check.

Remark 4.4 (Reading the class). The two coordinates of ω are the two worker outcomes; the coboundary diagonal is the freedom to relabel which token-state is “canonical.” The class is nonzero precisely because the two outcomes disagree on that relabelling by one token-flip — the re-entrant branch’s side effect. This is the categorical form of “a flat bundle is trivial iff its holonomy vanishes”: each partial handoff completes freely (by (L)), but the completions fail to glue into a global serialized agent by a single $\mathbb{Z}/2$ monodromy around the dispatch/return loop. It is the obstruction the Kodamai seed reads in Ethereum re-entrancy, now located as an explicit H^2 generator on a handoff category.

The composing regimes and the table. Theorem 4.2 is the proved core: the two re-entry rows of the table below are its $\varepsilon = 0, 1$ cases (the state-protected regime $\mathcal{K}_0$ carries a $\mathbb{Z}/2$ -torsor of factorization systems, $[\omega] = 0$; the unprotected regime $\mathcal{K}_1$ has none). Two *further* regimes round out the qualitative picture at the grade of a *computed* illustration, not part of the proved dichotomy: *independent supervisors* over read-only workers compose by the trivial (swap) law, $\mathcal{K} = \mathcal{C} \times \mathcal{D}$; and two supervisors over a shared worker pool can interleave nontrivially yet coherently — taking $\mathcal{C} = \mathbb{Z}/3, \mathcal{D} = \mathbb{Z}/2$ with the toggle reversing the rotation gives the dihedral matched pair $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D} = S_3$, a genuinely non-abelian joint agent that is neither a product nor obstructed, showing the criterion is non-vacuous. All four rows are machine-checked.

shared-resource topology	verdict	invariant
independent supervisors (read-only workers)	composes	$\mathcal{K} = \mathcal{C} \times \mathcal{D}$ (trivial law)
two supervisors, coherent nontrivial interleave	composes	$\mathcal{K} = S_3$, non-abelian join
re-entry, state-protected (lock)	composes	$\#SFS = 2, [\omega] = 0$
re-entry, unprotected (worker mutates state)	obstructed	$\#SFS = 0, [\omega] = \text{gen } \mathbb{Z}/2$

The single bit that flips the verdict is whether a worker’s outcome mutates the supervisor’s shared state. If it does, the distributive law becomes multivalued, (G) fails, and the obstruction to a joint agent is the nonzero $[\omega] \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D})$.

5. THE DEGREE AXIS

The container obstruction is best understood by contrast with how the multi-agent-systems literature uses cohomology, because the two are complementary and the difference is exactly one degree.

That literature places a cellular sheaf on the agents’ *communication* graph and measures low-degree obstructions. Global sections — degree H^0 — are consensus states and, in a game-theoretic sheaf, Nash equilibria [13]. A (nonlinear) sheaf Laplacian in degree H^1 measures identifiability of a multi-agent system [14], with related sheaf-theoretic formulations for resilient planning [15]. These are *state* questions on the communication graph: *do the agents currently agree, and can we recover their parameters?*

The container model asks a prior, *structural* question and finds it one degree up. The relevant graph is not the communication graph but the *handoff* category $\text{Sk}_{\mathcal{C}}$ — who may delegate to whom — and the coefficients are not a generic data sheaf but the agents’ own reply functor $\mathcal{D}$, read off from their container structure. The obstruction lives in H^2 , and it answers: *can these agents be fused into one consistent joint agent at all?* The contrast is summarised in the following table.

	sheaf-Laplacian MAS	container / directed-container
carrier	communication graph	handoff category $\text{Sk}_{\mathcal{C}}$
coefficients	data / strategy sheaf	reply functor $\mathcal{D}$ (from the container)
degree	H^0 (consensus), H^1 (identifiability)	H^2 (composability)
question	do agents agree / can we identify them?	do agents compose into a joint system?

The two programmes are not in competition; they answer different questions at different degrees. What is distinctive here is the source of the cohomology: everyone else applies generic sheaf/cochain machinery to a multi-agent graph, whereas the class $[\omega]$ is extracted from the agents’ *polynomial-functor* structure — the reply functor is the coefficient system, and the handoff category is the base. To our knowledge the composability obstruction has not previously been routed through directed-container structure.

The categorical-orchestration landscape. The sheaf-Laplacian programme is not the only neighbour. Three further categorical accounts of agentic composition appeared in 2026, and it is worth locating each, because none reaches for the structure this note turns on — a distributive law, a Zappa–Szép or semidirect product, or an obstruction to composing at all. Aberlé’s verified-implementation framework [5] composes systems along wiring diagrams by Kleisli composition, its only two-system construct the unobstructed parallel sum. Banu’s polynomial-functor treatment of agent architectures [17] composes skills by an *operad* of wiring diagrams — parallel and serial composition, with a trace — and its author checks explicitly that no distributive law, semidirect product, or Zappa–Szép structure appears: it is *parallel skill-assembly within one harness*. Waites’s typed “plumbing” language [18] models agent coordination in a symmetric monoidal *copy–discard* category, with session types compiling functorially onto the plumbing and a *traced*-monoidal inner/outer feedback loop separating an agent’s conversation history from its supervisory control; its characteristic failure mode is context drift under memory compaction, handled — tellingly — by a pragmatic serialisation gate rather than diagnosed as an obstruction class.

The contrast is sharpest against Waites, and it is instructive that both accounts reach for the word *serialisation*. Waites installs a serialisation gate as an engineering remedy; the present note

characterises exactly *when serialisation is impossible* — the unprotected-re-entry regime $\mathcal{K}_1$, where $[\omega]$ is the nonzero generator and no interleaving into a single serialized agent exists. Copy–discard symmetric monoidal structure with a mutex and Zappa–Szép/**Poly** structure with an H^2 class are two answers to adjacent questions: how to *wire* agents that share streams, versus whether two agents that share state can be *fused* into one at all. What unites these neighbours is the negative: operad, parallel-sum, and copy–discard composition are all unobstructed by construction, so none can express the composability question that is this note’s whole subject.

account	composition mechanism	two-interacting-agent obstruction
Aberlé [5]	free-monad implementations, wiring diagrams	none (parallel sum only)
Banu [17]	operad of wiring diagrams (parallel/serial, trace)	none (unobstructed operad)
Waites [18]	copy–discard SMC, traced feedback	none (mutex / serialisation gate)
this note	distributive law / Zappa–Szép $\mathcal{C} \bowtie \mathcal{D}$	$[\omega] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$

6. CONCLUSION

Read as containers, agents supply not only an interface vocabulary but a composition theory. The move that makes this productive is small and precise: composing two shared-resource orchestrations is not a functor along one topology but a Zappa–Szép product $\mathcal{C} \bowtie \mathcal{D}$ interleaving two, and a Zappa–Szép product can fail to exist. When it does, the failure is a single degree-two cohomology class on the handoff category, and the smallest instance of that failure is a recognisable engineering hazard — unprotected re-entrancy, a worker writing to the supervisor’s shared state mid-call — appearing as the nonzero generator of $H^2 \cong \mathbb{Z}/2$.

Status, honestly. The scaffolding is proved: Theorem 3.1 is Ahman–Uustalu (and machine-checked in the author’s Lean development); the Zappa–Szép-product-from-a-distributive-law construction underlying Theorem 3.2 is likewise Ahman–Uustalu’s [4], the author’s part being the (L) + (G) existence criterion; Theorem 3.3 is the author’s; and its cohomology is classical and cited, not reproved. The interface, free-monad-implementation, wiring-diagram-composition, and dependent-specification mechanism is Aberlé’s, cited and not reinvented. The contribution is the obstruction layer on top: the worked dichotomy of Section 4 (Theorem 4.2, $[\omega(\mathcal{K}_\varepsilon)] = \varepsilon \cdot \text{gen}$) is *proved* analytically, with a parametrized script as cross-check. Two things remain at a lower grade. The two extra table regimes (independent product, S_3 interleave) are *computed* illustrations, not part of the proved dichotomy. And the claim that a real orchestration framework literally instantiates $\mathcal{K}_\varepsilon$ is a *computed* reading, not a theorem about deployed systems: the models are minimal faithful abstractions of the supervisor–worker pattern, not literal encodings of any named framework.

The validation ask. The natural next step mirrors how compositional structure has been validated empirically elsewhere at Kodamai (the genetic-algorithm work, where migration-topology cases confirmed that composition determines dynamics): take one concrete orchestration framework, exhibit its handoff graph as a directed container $(S \triangleleft P, o, \downarrow, \oplus)$, and exhibit one of its concurrency patterns as an explicit distributive law λ — turning the starred dictionary row from computed into demonstrated on a system people actually run.

Open directions. Two stand out. First, a unifying reason the two cohomological pictures of Section 5 should meet: comonads generalise both directed containers and cellular sheaves [16], which suggests the H^2 composability obstruction and the H^0/H^1 sheaf obstructions are shadows of a single comonadic invariant — worth making precise. Second, the ∞ -analogue: replacing the handoff category by a higher category of handoffs and asking whether the obstruction tower extends, which is where a re-entrant tool-call that is itself an orchestration would live.

ACKNOWLEDGEMENTS

The agent-as-container and orchestration-as-monoidal-structure readings are due to Neil Ghani and the Kodamai programme, building on Spivak’s polynomial-functor framework; the higher-order-orchestration and harness-as-comonad rows are Ghani’s framing. The author thanks Neil Ghani and Robin Langer.

REFERENCES

- [1] N. Niu, D. I. Spivak. *Polynomial Functors: A Mathematical Theory of Interaction*. arXiv:2312.00990, 2023.
- [2] D. I. Spivak. *A Reference for Categorical Structures on Poly*. arXiv:2202.00534, 2022.
- [3] D. Ahman, T. Uustalu. *Directed Containers as Categories*. arXiv:1604.01187, 2016.
- [4] D. Ahman, T. Uustalu. *Distributive laws of directed containers*. Progress in Informatics, No. 10 (2013), pp. 3–18, DOI 10.2201/NIIP.2013.10.2. (Precursor: CMCS 2012.)
- [5] C. B. Aberlé. *Compositional Program Verification with Polynomial Functors in Dependent Type Theory*. arXiv:2604.01303, 2026.
- [6] MacBeth. *The pairwise Zappa–Szép criterion*. Kodamai note, 10 June 2026.
- [7] MacBeth. *The global-closure obstruction (G) is a cohomology class: the Zappa–Szép holonomy as $H^2(\mathrm{Sk}_C; \mathcal{D})$* . Kodamai note, 11 June 2026.
- [8] MacBeth. *The re-entrancy obstruction is the token-mutation bit: an analytic proof for orchestration Zappa–Szép products*. Kodamai note, 20 July 2026.
- [9] MacBeth. *The free monad on a container as a $\triangleleft$ -monoid (Free.lean); grafting-monoid unit and associativity laws, machine-checked*. Kodamai Lean development, July 2026.
- [10] R. Rosebrugh, R. J. Wood. *Distributive laws and factorization*. J. Pure Appl. Algebra 175 (2002), 327–353.
- [11] H.-J. Baues, G. Wirsching. *Cohomology of small categories*. J. Pure Appl. Algebra 38 (1985), 187–211.
- [12] M. Pirashvili. *Schreier theory of track categories*. arXiv:1512.03250.
- [13] M. Hernández, E. Sánchez-Soto. *A Sheaf Framework for Strategic Multi-Agent Systems: From Consensus to Nash Equilibria*. arXiv:2606.01663, 2026.
- [14] N. Anwer, H. Riess, M. Hale. *Multi-Agent System Identification with Nonlinear Sheaf Diffusion*. arXiv:2605.11204, 2026.
- [15] M. Hernández, E. Sánchez-Soto. *Sheaf-Theoretic Planning: A Categorical Foundation for Resilient Multi-Agent Autonomous Systems*. arXiv:2605.01879, 2026.
- [16] A. D. Fairbanks, K. Carlson, D. I. Spivak. *Comonads as Spaces*. arXiv:2607.15091, 2026.
- [17] B. Banu. *Harness Engineering as Categorical Architecture*. arXiv:2605.12239, 2026.
- [18] W. Waites. *A Typed Language for Agent Coordination and The Agent That Doesn’t Know Itself*. The n -Category Café, March 2026. https://golem.ph.utexas.edu/category/2026/03/a_typed_language_for_agent_coordination.html, https://golem.ph.utexas.edu/category/2026/03/the_agent_that_doesnt_know_its.html.

KODAMAI

Email address: neil@kodamai.com